

TP Spark SQL : Analyse du Système de Location de Vélos

Big Data 2025

Rapport de Travaux Pratiques

Étudiant : Zakaria El guazzar

Groupe : SDIA

Date : 6 décembre 2025

Encadré par : M. Abdelmajid BOUSSELHAM

Institution : Enset

Table des matières

1	Introduction	3
2	Contexte et Objectifs	3
2.1	Contexte	3
2.2	Objectifs	3
3	Environnement Technique	3
3.1	Technologies Utilisées	3
3.2	Architecture Déployée	3
4	Description des Données	4
4.1	Fichier Source	4
4.2	Structure des Données	4
5	Démarche Suivie	5
5.1	Préparation du Projet Maven	5
5.2	Code Principal - Application Local	5
5.3	Code Principal - Application HDFS	6
5.4	Commandes d'Exécution	6
6	Requêtes et Résultats	7
6.1	Exploration Initiale	7
6.2	Requêtes de Base	7
6.3	Requêtes d'Agrégation	7
6.4	Analyse Temporelle	8
6.5	Analyse des Utilisateurs	8
7	Résultats Détaillés	9
7.1	Tableau Synthétique des Résultats	9
7.2	Répartition par Station	9
7.3	Répartition par Groupe d'Âge	10
8	Interprétation des Résultats	10
8.1	Popularité des Stations	10
8.2	Comportement Temporel	10
8.3	Profil des Utilisateurs	10
8.4	Implications Opérationnelles	10
9	Difficultés Rencontrées et Solutions	10
9.1	Problème 1 : Connexion HDFS	10
9.2	Problème 2 : Packaging des Dépendances	11
9.3	Problème 3 : Persistance HDFS	11
10	Conclusion	11
10.1	Acquis Techniques	11
10.2	Acquis Analytiques	11
10.3	Perspectives	12

11 Annexes	12
11.1 Structure du Projet	12
11.2 Résultats d'Exécution	12
11.3 Commandes Utiles pour le Monitoring	12

1 Introduction

Ce rapport présente les travaux pratiques réalisés sur **Spark SQL** dans le cadre de l'analyse d'un système de location de vélos en libre-service. L'objectif principal était de manipuler un jeu de données transactionnel à l'aide de requêtes SQL exécutées dans un environnement Spark, déployé localement et en cluster HDFS.

L'analyse couvre plusieurs aspects du système :

- Comportement des utilisateurs
- Popularité des stations
- Analyse temporelle (heures de pointe)
- Performance globale du système

2 Contexte et Objectifs

2.1 Contexte

Une ville souhaite analyser l'utilisation de son système de vélos en libre-service pour optimiser les ressources, améliorer le service aux usagers et planifier les maintenances.

2.2 Objectifs

Les objectifs techniques étaient :

1. Charger et explorer les données avec Spark DataFrame
2. Créer une vue temporaire SQL
3. Exécuter des requêtes analytiques complexes
4. Déployer l'application sur un cluster Spark/HDFS
5. Visualiser et interpréter les résultats

3 Environnement Technique

3.1 Technologies Utilisées

Technologie	Version
Apache Spark	4.0.0
Hadoop HDFS	3.3.6
Java	21
Maven	3.9+
Docker	Latest
Docker Compose	v2

TABLE 1 – Stack technologique utilisée

3.2 Architecture Déployée

L'infrastructure complète a été conteneurisée avec Docker Compose :

```

1 services:
2   namenode:
3     image: apache/hadoop:3.3.6
4     ports: ["9870:9870", "8020:8020"]
5
6   datanode:
7     image: apache/hadoop:3.3.6
8
9   resourcemanager:
10    image: apache/hadoop:3.3.6
11    ports: ["8088:8088"]
12
13  nodemanager:
14    image: apache/hadoop:3.3.6
15
16  spark-master:
17    image: spark:latest
18    ports: ["7077:7077", "8080:8080"]
19
20  spark-worker-1:
21    image: spark:latest
22

```

Listing 1 – Architecture Docker Compose

4 Description des Données

4.1 Fichier Source

- **Nom** : bike_sharing.csv
- **Lignes** : 100 locations
- **Colonnes** : 10 attributs

4.2 Structure des Données

Colonne	Description
rental_id	Identifiant unique de location
user_id	Identifiant unique de l'utilisateur
age	Âge de l'utilisateur
gender	Genre (M/F)
start_time	Heure de début (timestamp)
end_time	Heure de fin (timestamp)
start_station	Station de départ
end_station	Station d'arrivée
duration_minutes	Durée en minutes
price	Prix en dollars

TABLE 2 – Structure du dataset bike_sharing.csv

5 Démarche Suivie

5.1 Préparation du Projet Maven

```

1 <dependencies>
2   <!-- Spark Core -->
3   <dependency>
4     <groupId>org.apache.spark</groupId>
5     <artifactId>spark-core_2.13</artifactId>
6     <version>${spark.version}</version>
7   </dependency>
8
9   <!-- Spark SQL -->
10  <dependency>
11    <groupId>org.apache.spark</groupId>
12    <artifactId>spark-sql_2.13</artifactId>
13    <version>${spark.version}</version>
14  </dependency>
15 </dependencies>

```

Listing 2 – Configuration Maven (pom.xml)

5.2 Code Principal - Application Local

```

1 import org.apache.spark.sql.*;
2
3 public class BikeSharingAppLocal {
4   public static void main(String[] args) {
5     // Initialisation Spark
6     SparkSession spark = SparkSession.builder()
7       .appName("BikeSharingAnalysis")
8       .master("local[*]")
9       .getOrCreate();
10
11     // Chargement CSV
12     Dataset<Row> df = spark.read()
13       .option("header", "true")
14       .option("inferSchema", "true")
15       .csv("bike_sharing.csv");
16
17     // Exploration
18     df.printSchema();
19     df.show(5);
20     System.out.println("Number of rentals: " + df.count());
21
22     // Cr ation vue SQL
23     df.createOrReplaceTempView("bike_rentals_view");
24
25     // Requ te 1: Locations > 30 minutes
26     Dataset<Row> longRentals = spark.sql(
27       "
28       );
29
30     spark.stop();

```

```

31 }
32 }

```

Listing 3 – Classe BikeSharingAppLocal.java

5.3 Code Principal - Application HDFS

```

1 import org.apache.spark.sql.*;
2
3 public class BikeSharingAppHdfs {
4     public static void main(String[] args) {
5         // Configuration HDFS
6         SparkConf conf = new SparkConf()
7             .setAppName("BikeSharingAnalysisHDFS")
8             .set("spark.hadoop.fs.defaultFS", "hdfs://namenode:8020"
9         );
10
11         SparkSession spark = SparkSession.builder()
12             .config(conf)
13             .getOrCreate();
14
15         // Chargement depuis HDFS
16         String hdfsPath = "hdfs://namenode:8020/bike_sharing.csv";
17         Dataset<Row> df = spark.read()
18             .option("header", "true")
19             .option("inferSchema", "true")
20             .csv(hdfsPath);
21
22         // Ex cution des requ tes...
23         spark.stop();
24     }
25 }

```

Listing 4 – Classe BikeSharingAppHdfs.java

5.4 Commandes d'Exécution

```

1 # 1. Compilation et packaging
2 mvn clean compile package
3
4 # 2. Lancement de l'infrastructure
5 docker compose up -d
6
7 # 3. Copie des fichiers
8 docker cp target/Lab4_spark_sql-1.0-SNAPSHOT.jar spark-master:/opt/spark
9 /
10 docker cp bike_sharing.csv namenode:/data
11
12 # 4. Transfert vers HDFS
13 docker exec -it namenode bash
14 hdfs dfs -put /data/bike_sharing.csv /
15
16 # 5. Ex cution Spark
17 /opt/spark/bin/spark-submit --master spark://spark-master:7077
18     Lab4_spark_sql-1.0-SNAPSHOT.jar

```

Listing 5 – Commandes d'exécution

6 Requêtes et Résultats

6.1 Exploration Initiale

```
1 -- Schema
2 df.printSchema();
3
4 -- Premi res lignes
5 df.show(5);
6
7 -- Nombre total
8 SELECT COUNT(*) FROM bike_rentals_view;
9 -- R sultat: 100 locations
```

Listing 6 – Exploration des données

6.2 Requêtes de Base

```
1 -- 1. Locations > 30 minutes
2 SELECT * FROM bike_rentals_view
3 WHERE duration_minutes > 30;
4 -- R sultat: 58 locations
5
6 -- 2. Locations depuis Station A
7 SELECT * FROM bike_rentals_view
8 WHERE start_station = 'Station A';
9 -- R sultat: 17 locations
10
11 -- 3. Revenu total
12 SELECT SUM(price) AS total_revenue
13 FROM bike_rentals_view;
14 -- R sultat: 871.0 $
```

Listing 7 – Requêtes SQL de base

6.3 Requêtes d'Agrégation

```
1 -- 1. Locations par station
2 SELECT start_station, COUNT(*) AS rental_count
3 FROM bike_rentals_view
4 GROUP BY start_station
5 ORDER BY rental_count DESC;
6
7 -- 2. Dur e moyenne par station
8 SELECT start_station, AVG(duration_minutes) AS avg_duration
9 FROM bike_rentals_view
10 GROUP BY start_station;
11
```

```
12 -- 3. Station la plus utilis e
13 SELECT start_station, COUNT(*) AS rental_count
14 FROM bike_rentals_view
15 GROUP BY start_station
16 ORDER BY rental_count DESC
17 LIMIT 1;
18 -- R sultat: Station B (27 locations)
```

Listing 8 – Requêtes d'agrégation

6.4 Analyse Temporelle

```
1 -- 1. Extraction de l'heure
2 SELECT *, HOUR(start_time) AS rental_hour
3 FROM bike_rentals_view;
4
5 -- 2. Locations par heure
6 SELECT HOUR(start_time) AS rental_hour,
7        COUNT(*) AS rental_count
8 FROM bike_rentals_view
9 GROUP BY rental_hour
10 ORDER BY rental_hour;
11
12 -- 3. Station populaire le matin (7h-12h)
13 SELECT start_station, COUNT(*) AS rental_count
14 FROM bike_rentals_view
15 WHERE HOUR(start_time) BETWEEN 7 AND 12
16 GROUP BY start_station
17 ORDER BY rental_count DESC
18 LIMIT 1;
19 -- R sultat: Station E (8 locations)
```

Listing 9 – Requêtes temporelles

6.5 Analyse des Utilisateurs

```
1 -- 1. ge moyen
2 SELECT AVG(age) AS average_age
3 FROM bike_rentals_view;
4 -- R sultat: 41.91 ans
5
6 -- 2. R partition par genre
7 SELECT gender, COUNT(*) AS user_count
8 FROM bike_rentals_view
9 GROUP BY gender;
10
11 -- 3. Groupe d' ge le plus actif
12 SELECT
13     CASE
14         WHEN age BETWEEN 18 AND 30 THEN '18-30'
15         WHEN age BETWEEN 31 AND 40 THEN '31-40'
16         WHEN age BETWEEN 41 AND 50 THEN '41-50'
17         WHEN age >= 51 THEN '51+'
18     END AS age_group,
```

```

19      COUNT(*) AS rental_count
20 FROM bike_rentals_view
21 GROUP BY age_group
22 ORDER BY rental_count DESC
23 LIMIT 1;
24 -- R sultat: 51+ (34 locations)

```

Listing 10 – Analyse des utilisateurs

7 Résultats Détaillés

7.1 Tableau Synthétique des Résultats

Métrique	Description	Valeur
Locations totales	Nombre de transactions	100
Revenu total	Somme des prix	871.0 \$
Durée moyenne	Moyenne générale	34.75 min
Âge moyen	Moyenne des âges	41.91 ans
Station la plus utilisée	Station B	27 locations
Station matinale	Station E (7h-12h)	8 locations
Pic horaire	17h	7 locations
Groupe d'âge	51+ ans	34 locations

TABLE 3 – Synthèse des résultats principaux

7.2 Répartition par Station

Station	Nombre de Locations	Durée Moyenne (min)
Station A	17	31.94
Station B	27	36.74
Station C	14	33.00
Station D	19	33.79
Station E	23	36.74

TABLE 4 – Performance par station de départ

7.3 Répartition par Groupe d'Âge

Groupe d'Âge	Nombre de Locations
18-30 ans	24
31-40 ans	21
41-50 ans	21
51+ ans	34

TABLE 5 – Répartition par tranche d'âge

8 Interprétation des Résultats

8.1 Popularité des Stations

- **Station B** est la plus fréquentée globalement (27 locations)
- La **Station E** domine le créneau matinal (7h-12h)
- Les durées moyennes sont similaires sur toutes les stations (32-37 min)

8.2 Comportement Temporel

- **Pic d'utilisation** : 17h (7 locations)
- **Période calme** : Nuit (0h-6h) et après-midi tardif
- Distribution relativement uniforme sur la journée

8.3 Profil des Utilisateurs

- **Âge moyen** : 41.91 ans
- **Groupe dominant** : 51+ ans (34% des locations)
- **Répartition genre** : Presque équilibrée (52% F, 48% M)

8.4 Implications Opérationnelles

1. **Répartition des vélos** : Renforcer Station B et Station E
2. **Maintenance** : Programmer pendant les heures creuses
3. **Marketing** : Cibler les 50+ ans et les trajets matinaux
4. **Tarification** : Possibilité de modulation horaire

9 Difficultés Rencontrées et Solutions

9.1 Problème 1 : Connexion HDFS

- **Symptôme** : Spark ne trouve pas le fichier sur HDFS
- **Cause** : Configuration incorrecte de `fs.defaultFS`
- **Solution** :

```
1 SparkConf conf = new SparkConf()  
2     .set("spark.hadoop.fs.defaultFS", "hdfs://namenode:8020");  
3
```

9.2 Problème 2 : Packaging des Dépendances

- **Symptôme** : ClassNotFoundException à l'exécution
- **Cause** : Dépendances Spark non incluses dans le JAR
- **Solution** : Utilisation de maven-assembly-plugin

```
1 <plugin>  
2     <artifactId>maven-assembly-plugin</artifactId>  
3     <configuration>  
4         <descriptorRefs>  
5             <descriptorRef>jar-with-dependencies</descriptorRef>  
6         </descriptorRefs>  
7     </configuration>  
8 </plugin>  
9
```

9.3 Problème 3 : Persistance HDFS

- **Symptôme** : Données perdues au redémarrage
- **Cause** : Volumes Docker non persistants
- **Solution** : Montage de volumes locaux

```
1 volumes:  
2 - ./volumes/namenode/name:/hadoop/dfs/name  
3 - ./volumes/datanode/data:/hadoop/dfs/data  
4
```

10 Conclusion

Ce TP a permis de maîtriser plusieurs aspects essentiels du Big Data avec Spark :

10.1 Acquis Techniques

- Déploiement d'un cluster Spark/HDFS conteneurisé
- Programmation Spark SQL en Java
- Analyse exploratoire de données transactionnelles
- Packaging et déploiement d'applications Spark

10.2 Acquis Analytiques

- Identification des patterns d'utilisation
- Analyse temporelle et segmentation utilisateur
- Production d'indicateurs opérationnels
- Recommandations basées sur les données

10.3 Perspectives

1. **Étendre l'analyse** : Ajouter des données météo, événements
2. **Prédiction** : Modèles ML pour prévoir la demande
3. **Temps réel** : Dashboard de monitoring
4. **Optimisation** : Algorithme de répartition dynamique

11 Annexes

11.1 Structure du Projet

```
1 Lab4_spark_sql/  
2     src/main/java/  
3         BikeSharingAppLocal.java  
4         BikeSharingAppHdfs.java  
5     pom.xml  
6     docker-compose.yaml  
7     bike_sharing.csv  
8     target/  
9         Lab4_spark_sql-1.0-SNAPSHOT.jar
```

11.2 Résultats d'Exécution

```
1 Schema:  
2 root  
3 |-- rental_id: integer  
4 |-- user_id: integer  
5 |-- age: integer  
6 |-- gender: string  
7 |-- start_time: timestamp  
8 |-- end_time: timestamp  
9 |-- start_station: string  
10 |-- end_station: string  
11 |-- duration_minutes: integer  
12 |-- price: double  
13  
14 Nombre de locations: 100  
15 Revenu total: 871.0  
16 Station la plus utilis e: Station B (27 locations)  
17 Groupe d' ge dominant: 51+ (34 locations)
```

Listing 11 – Extrait des résultats d'exécution

11.3 Commandes Utiles pour le Monitoring

```
1 # V rification des services  
2 docker ps  
3  
4 # Logs Spark Master  
5 docker logs spark-master
```

```
6
7 # Interface Web HDFS
8 http://localhost:9870
9
10 # Interface Web Spark
11 http://localhost:8080
12
13 # Interface Web YARN
14 http://localhost:8088
15
16 # Liste fichiers HDFS
17 docker exec namenode hdfs dfs -ls /
```

Références

- [1] Apache Spark Documentation. *Spark SQL and DataFrames*. 2024. <https://spark.apache.org/docs/latest/sql-programming-guide.html>
- [2] Apache Hadoop Documentation. *HDFS Architecture*. 2024. <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>
- [3] Docker Documentation. *Docker Compose*. 2024. <https://docs.docker.com/compose/>

Rédigé et exécuté par :
Zakaria El guazzar
Étudiant en Big Data
6 décembre 2025